Lecture 12. Iteration

For loops, while loops, map functions, failure handling, multiple arguments

Contents

12.1 Iteration **2**

12.2 For loops **2**
12.2.1 Variations 2

12.3 For loops versus functionals **4**

12.4 The map functions **4**
12.4.1 Shortcuts 5
12.4.2 Base R 5

12.5 Dealing with failure **6**

12.6 Mapping over multiple arguments **6**

12.7 Walk, reduce and accumulate **7**

12.1 Iteration

Motivation. Functions removed duplicated code by naming a computation; iteration removes it by repeating a computation over many inputs, such as every column of a data frame or every group of a data set. R offers two styles, explicit loops and functional programming, and both are worth knowing.

Reducing duplication has three benefits: the intent of the code is easier to see, requirements change in one place, and there are fewer bugs because each line is used in more places. Lecture 10 used functions; this lecture uses **iteration**: *imperative* for and while loops, which make iteration explicit, and *functional* tools (`purrr`), which hide the boilerplate.

12.2 For loops

To compute the median of each column of

```
df <- tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))
```

copying `median(df$a)` four times breaks the rule of thumb. A for loop:

```
output <- vector("double", ncol(df))   # 1. output
for (i in seq_along(df)) {             # 2. sequence
  output[[i]] <- median(df[[i]])       # 3. body
}
output
```

```
## [1] -0.24576245 -0.28730721 -0.05669771  0.14426335
```

Every loop has three components.

1. **Output**: allocate enough space before the loop with `vector(type, length)`. Growing a vector with `c()` at each step is very slow, because the whole vector is copied every time.
2. **Sequence**: `i in seq_along(df)` gives a different value of `i` on each run. `seq_along()` is a safe version of `1:length()`: for a zero-length vector it gives an empty sequence, whereas `1:length(y)` gives `c(1, 0)` and runs the body on nonsense indices.
3. **Body**: the code that does the work, run once per value of `i`. Use `[[ ]]` even for atomic vectors, to make clear that one element is handled.

```
y <- vector("double", 0)
seq_along(y)
```

```
## integer(0)
```

```
1:length(y)
```

```
## [1] 1 0
```

12.2.1 Variations

Modifying an existing object. Rescaling every column of `df` (Lecture 10):

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
for (i in seq_along(df)) {
  df[[i]] <- rescale01(df[[i]])
```

```
}
```

Here the output is `df` itself, so no new object is needed.

Looping patterns. Besides looping over indices, one can loop over the elements, `for (x in xs)`, which is awkward for saving output, or over names, `for (nm in names(xs))`, accessing values with `x[[nm]]`. Looping over indices is the most general: from the position one gets both the name and the value:

```
results <- vector("list", length(x))
names(results) <- names(x)
for (i in seq_along(x)) {
  name <- names(x)[[i]]
  value <- x[[i]]
}
```

Unknown output length. If each iteration produces a vector of random length, do not grow the result with `c()` (quadratic time). Store the pieces in a list and combine once at the end, with `unlist()` (or `purrr::flatten_dbl()`) for vectors, `paste(collapse = "")` for strings, and `dplyr::bind_rows()` for data frames:

```
means <- c(0, 1, 2)
out <- vector("list", length(means))
for (i in seq_along(means)) {
  n <- sample(100, 1)
  out[[i]] <- rnorm(n, means[[i]])
}
str(out)
```

```
## List of 3
##  $ : num [1:38] 0.255 -0.553 1.405 -0.795 -1.567 ...
##  $ : num [1:17] 0.91 2.92 2.3 1.75 1.56 ...
##  $ : num [1:14] 3.215 0.722 2.748 5.392 3.619 ...
```

```
str(unlist(out))
```

```
##  num [1:69] 0.255 -0.553 1.405 -0.795 -1.567 ...
```

Unknown sequence length: while loops. A `while` loop runs while a condition is `TRUE`; every for loop can be written as a while loop but not vice versa. Simulations often need it, e.g. the number of coin flips until three heads in a row:

```
flip <- function() sample(c("T", "H"), 1)
flips <- 0
nheads <- 0
while (nheads < 3) {
  if (flip() == "H") nheads <- nheads + 1 else nheads <- 0
  flips <- flips + 1
}
flips
```

```
## [1] 18
```

How many flips on average? [Supplement]

Let e_k be the expected number of further flips when the current run of heads has length k. Then $e_3 = 0$ and $e_k = 1 + \frac{1}{2}e_{k+1} + \frac{1}{2}e_0$ for $k = 0, 1, 2$. Solving gives $e_0 = 14$. A simulation agrees:

```
flips_until_3 <- function() {
  flips <- 0; nheads <- 0
  while (nheads < 3) {
```

```
    if (flip() == "H") nheads <- nheads + 1 else nheads <- 0
    flips <- flips + 1
  }
  flips
}
mean(replicate(5000, flips_until_3()))
```

```
## [1] 13.9762
```

12.3 For loops versus functionals

Motivation. Loops written for each new summary differ only in the function they apply. Passing that function as an argument removes the remaining duplication; this idea, functions that take functions, is what purrr is built on.

Column means, medians and standard deviations each need a nearly identical loop. The only difference is the function called, so make it an argument:

```
col_summary <- function(df, fun) {
  out <- vector("double", length(df))
  for (i in seq_along(df)) {
    out[i] <- fun(df[[i]])
  }
  out
}
col_summary(df, median)
```

```
## [1] 0.6111823 0.4011430 0.4280338 0.5813702
```

```
col_summary(df, sd)
```

```
## [1] 0.3117016 0.2993812 0.3078818 0.2728141
```

A function that takes another function as input is a **functional**. Base R's `apply()`, `lapply()`, `tapply()` are functionals; `purrr` provides a consistent family.

12.4 The map functions

Looping over a vector, doing something to each element and saving the results is so common that purrr has a function for each output type:

function	output
`map()`	list
`map_lgl()`	logical vector
`map_int()`	integer vector
`map_dbl()`	double vector
`map_chr()`	character vector

```
df %>% map_dbl(mean)
```

```
##         a         b         c         d
## 0.5566313 0.4287933 0.4083740 0.5836067
```

```
df %>% map_dbl(sd)
```

```
##         a         b         c         d
## 0.3117016 0.2993812 0.3078818 0.2728141
```

Compared with `col_summary()`, map functions are written in C (fast), pass extra arguments through `...` (`map_dbl(df, mean, trim = 0.5)`), and keep names. The focus moves from the bookkeeping of the loop to the operation performed; this does not make loops "slow" (that is a myth when output is preallocated), it makes code clearer.

12.4.1 Shortcuts

To fit a linear model to each cylinder group of `mtcars`, `split()` the data into a list of data frames and map a function over it. Anonymous functions can be written in a one-sided formula, where `.` (or `.x`) is the current element:

```
models <- mtcars %>%
  split(.$cyl) %>%
  map(~lm(mpg ~ wt, data = .))
models %>%
  map(summary) %>%
  map_dbl(~.x$r.squared)
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

Extracting a named component is so common that a string suffices, and an integer selects by position:

```
models %>% map(summary) %>% map_dbl("r.squared")
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

```
x <- list(list(1, 2, 3), list(4, 5, 6), list(7, 8, 9))
x %>% map_dbl(2)
```

```
## [1] 2 5 8
```

12.4.2 Base R

`lapply()` is essentially `map()`. `sapply()` simplifies the output automatically, which is convenient interactively but dangerous in functions, because the output type depends on the data:

```
threshold <- function(x, cutoff = 0.8) x[x > cutoff]
x1 <- list(c(0.27, 0.37, 0.57, 0.91, 0.20), c(0.90, 0.94, 0.66, 0.63, 0.06),
           c(0.21, 0.18, 0.69, 0.38, 0.77))
x2 <- list(c(0.50, 0.72, 0.99, 0.38, 0.78), c(0.93, 0.21, 0.65, 0.13, 0.27),
           c(0.39, 0.01, 0.38, 0.87, 0.34))
x1 %>% sapply(threshold) %>% str()
```

```
## List of 3
##  $ : num 0.91
##  $ : num [1:2] 0.9 0.94
##  $ : num(0)
```

```
x2 %>% sapply(threshold) %>% str()
```

```
##  num [1:3] 0.99 0.93 0.87
```

The first returns a list, the second a numeric vector. `vapply()` is a safe alternative that requires the output type, `vapply(df, is.numeric, logical(1))`.

12.5 Dealing with failure

Motivation. When mapping over many inputs, one failure stops everything and loses all the results that worked. purrr's adverbs turn errors into data, so failures can be inspected after the run.

`safely()` wraps a function so that it never throws an error; it returns a list with `result` and `error`, one of which is `NULL`:

```
safe_log <- safely(log)
str(safe_log(10))
```

```
## List of 2
##  $ result: num 2.3
##  $ error : NULL
```

```
x <- list(1, 10, "a")
y <- x %>% map(safely(log)) %>% transpose()
is_ok <- y$error %>% map_lgl(is_null)
x[!is_ok]
```

```
## [[1]]
## [1] "a"
```

```
y$result[is_ok] %>% flatten_dbl()
```

```
## [1] 0.000000 2.302585
```

`transpose()` turns a list of pairs into a pair of lists. `possibly()` returns a default value on error, `x %>% map_dbl(possibly(log, NA_real_))`; `quietly()` captures printed output, messages and warnings instead of errors.

12.6 Mapping over multiple arguments

To simulate normal samples with varying means, `map(mu, rnorm, n = 5)` varies one argument. To vary both mean and standard deviation, `map2()` iterates over two vectors in parallel; `pmap()` takes a list of any number of arguments (named, to match them safely). A data frame is a list of equal-length columns, so a tibble of parameters drives `pmap()` directly:

```
mu <- list(5, 10, -3)
sigma <- list(1, 5, 10)
map2(mu, sigma, rnorm, n = 5) %>% str()
```

```
## List of 3
##  $ : num [1:5] 5.45 4.52 4.16 5.48 2.03
##  $ : num [1:5] 17.59 1.02 10.05 8.09 6.07
##  $ : num [1:5] -6.78 7.76 14.72 -6.58 -11.58
```

```
params <- tribble(
  ~mean, ~sd, ~n,
      5,   1,  1,
     10,   5,  3,
     -3,  10,  5
)
params %>% pmap(rnorm) %>% str()
```

```
## List of 3
##  $ : num 3.99
##  $ : num [1:3] 13.2 14.4 12.6
##  $ : num [1:5] -8.43 4.12 -12.89 5.54 -15.36
```

`map2()` is just a loop in disguise: it allocates a list and calls `f(x[[i]], y[[i]], ...)` for each `i`. To vary the function itself as well, combine `map2()` with `exec()`, e.g. `map2(c("runif", "rnorm",`

`"rpois"), param, function(fn, args) exec(fn, !!!args))` (the older `invoke_map()` is deprecated).

12.7 Walk, reduce and accumulate

walk() is used for side effects (printing, saving files) instead of return values; `walk()`, `walk2()`, `pwalk()` return their first argument invisibly, e.g. `pwalk(list(paths, plot = plots), ggsave, path = tempdir())` saves a list of plots to files.

reduce() combines a list into one value by applying a two-argument function repeatedly. To join a list of data frames on a common key, `dfs %>% reduce(full_join)`; to find values common to all vectors, `vs %>% reduce(intersect)`. **accumulate()** keeps the intermediate results:

```
x <- sample(10)
x
```

```
##  [1]  1  7  8  2  6 10  3  9  4  5
```

```
x %>% accumulate(`+`)
```

```
##  [1]  1  8 16 18 24 34 37 46 50 55
```